Imports & Path Verification (Run this first)

In [ ]:
```python
import os
import glob
import numpy as np
import cv2
from pathlib import Path
from ultralytics import YOLO

VAL_IMAGES_DIR = "/home/lamacpp/Documents/car_defect_detection/data/processe
VAL_LABELS_DIR = "/home/lamacpp/Documents/car_defect_detection/data/processe
MODEL_PATH = "/home/lamacpp/Documents/car_defect_detection/runs/segment/stag

# Verify paths exist
for p in [VAL_IMAGES_DIR, VAL_LABELS_DIR, MODEL_PATH]:
    print(f"✅ Found: {p}" if os.path.exists(p) else f"❌ MISSING: {p}")
```

Extract Ground Truth for Rare Classes

In [ ]:
```python
# Target classes: 5 (corrosion) and 6 (disjoint_part)
TARGET_CLASSES = {5: "corrosion", 6: "disjoint_part"}

gt_targets = {}

label_files = glob.glob(os.path.join(VAL_LABELS_DIR, "*.txt"))
print(f"Scanning {len(label_files)} validation label files...")

for label_file in label_files:
    img_name = os.path.basename(label_file).replace(".txt", "")

    # Find the actual image file (handling .jpg, .png, etc.)
    img_path = None
    for ext in [".jpg", ".jpeg", ".png", ".JPG", ".PNG"]:
        test_path = os.path.join(VAL_IMAGES_DIR, img_name + ext)
        if os.path.exists(test_path):
            img_path = test_path
            break

    if not img_path:
        continue

    img_h, img_w = cv2.imread(img_path).shape[:2]

    with open(label_file, "r") as f:
        lines = f.readlines()

    targets = []
    for line in lines:
        parts = line.strip().split()
        if len(parts) >= 5:
```

```
            cls_id = int(parts[0])
            if cls_id in TARGET_CLASSES:
                coords = list(map(float, parts[1:]))
                xs = coords[0::2]
                ys = coords[1::2]
                # True bounding box = min/max of all polygon vertices
                abs_x1 = int(min(xs) * img_w)
                abs_y1 = int(min(ys) * img_h)
                abs_x2 = int(max(xs) * img_w)
                abs_y2 = int(max(ys) * img_h)

                targets.append({
                    'class_id': cls_id,
                    'class_name': TARGET_CLASSES[cls_id],
                    'bbox_abs': [abs_x1, abs_y1, abs_x2, abs_y2]
                })

    if targets:
        gt_targets[img_name] = {'targets': targets, 'path': img_path, 'img_w

print(f"Found {len(gt_targets)} validation images containing corrosion or di
```

Run Inference with Ultra-Low Threshold

In [ ]:
```
# Load the model
model = YOLO(MODEL_PATH)

# We use a very low conf threshold (0.01) and high IoU (0.99)
# This is just to find candidates. We will do the true "API bypass" raw logi
results = model.predict(
    source=list([v['path'] for k, v in gt_targets.items()]),
    conf=0.01,
    iou=0.99,
    save=False,
    verbose=False
)

print(f"Ran inference on {len(results)} images.")
```

Identify the Failures

In [ ]:
```
missed_cases = []

for i, (img_name, gt_data) in enumerate(gt_targets.items()):
    res = results[i]

    # Extract model predictions
    if res.boxes is None or len(res.boxes) == 0:
        preds = []
    else:
        preds = res.boxes
```

```
    for gt in gt_data['targets']:
        gt_box = np.array(gt['bbox_abs'])
        gt_cls = gt['class_id']

        detected = False

        if len(preds) > 0:
            pred_boxes = preds.xyxy.cpu().numpy()
            pred_classes = preds.cls.cpu().numpy().astype(int)

            for pb, pc in zip(pred_boxes, pred_classes):
                # Only compare if the predicted class matches the ground tru
                if pc == gt_cls:
                    # Calculate IoU
                    x1 = max(gt_box[0], pb[0])
                    y1 = max(gt_box[1], pb[1])
                    x2 = min(gt_box[2], pb[2])
                    y2 = min(gt_box[3], pb[3])

                    inter = max(0, x2 - x1) * max(0, y2 - y1)
                    area_gt = (gt_box[2] - gt_box[0]) * (gt_box[3] - gt_box[
                    area_pb = (pb[2] - pb[0]) * (pb[3] - pb[1])
                    union = area_gt + area_pb - inter

                    iou = inter / union if union > 0 else 0

                    # If IoU > 0.3, the model "saw" it (even if it threw it
                    if iou > 0.3:
                        detected = True
                        break

        if not detected:
            missed_cases.append({
                'image': img_name,
                'path': gt_data['path'],
                'class_name': gt['class_name'],
                'gt_bbox': gt['bbox_abs'],
                'img_shape': (gt_data['img_h'], gt_data['img_w'])
            })

print(f"❌ Total missed instances (IoU < 0.3): {len(missed_cases)}")
for case in missed_cases:
    print(f" - {case['image']} | Class: {case['class_name']} | BBox: {case['
```

Introspect the Raw Output

In [ ]:
```
# import torch
# import cv2

# # Subject: first missed case (tiny corrosion)
# case = missed_cases[0]
# img = cv2.imread(case['path'])
# print("Subject image:", case['image'], "| original shape (H,W,C):", img.sh
```

```
# # Simple resize to 640x640 (exact letterbox mapping comes in Cell 7)
# img_resized = cv2.resize(img, (640, 640))
# img_rgb = img_resized[:, :, ::-1].copy()
# t = torch.from_numpy(img_rgb).permute(2, 0, 1).float().unsqueeze(0) / 255.
# t = t.to('cuda')

# net = model.model

# def describe(obj, prefix=""):
#     if torch.is_tensor(obj):
#         print(f"{prefix}tensor shape={tuple(obj.shape)} dtype={obj.dtype}"
#     elif isinstance(obj, (list, tuple)):
#         print(f"{prefix}{type(obj).__name__} of len {len(obj)}")
#         for i, o in enumerate(obj):
#             describe(o, prefix + f"  [{i}] ")
#     elif isinstance(obj, dict):
#         print(f"{prefix}dict keys={list(obj.keys())}")
#         for k, o in obj.items():
#             describe(o, prefix + f"  [{k}] ")
#     else:
#         print(f"{prefix}{type(obj).__name__}: {obj}")

# net.eval()
# with torch.no_grad():
#     out_eval = net(t)
# print("=== EVAL mode raw output ===")
# describe(out_eval)

# head = net.model[-1]
# print("=== Head introspection ===")
# print("Head type:", type(head).__name__)
# for attr in ["nc", "nm", "no", "stride", "reg_max", "end2end"]:
#     if hasattr(head, attr):
#         print(f"  head.{attr} =", getattr(head, attr))
# print("Head submodules:", [name for name, _ in head.named_children()])

# net.eval()  # restore
```

X-ray Helpers + CONTROL Case

In [ ]:

```
import torch, cv2

net = model.model
net.eval()
head = net.model[-1]
FEAT_IDX = [16, 19, 22]  # feature layers feeding the head

def letterbox(img, new_shape=640):
    h, w = img.shape[:2]
    scale = min(new_shape / w, new_shape / h)
    nw, nh = int(round(w * scale)), int(round(h * scale))
    dw, dh = (new_shape - nw) / 2, (new_shape - nh) / 2
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_LINEAR)
```

```
    top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
    left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
    padded = cv2.copyMakeBorder(resized, top, bottom, left, right,
                                cv2.BORDER_CONSTANT, value=(114, 114, 114))
    return padded, scale, dw, dh

def to_tensor(img_bgr):
    rgb = img_bgr[:, :, ::-1].copy()
    t = torch.from_numpy(rgb).permute(2, 0, 1).float().unsqueeze(0) / 255.0
    return t.to('cuda')

def get_feats(t):
    # Replicates ultralytics' _predict_once wiring (Concat modules need save
    feats, y, x = {}, [], t
    for m in net.model:
        if m.i == 23:
            break
        x = m(x) if m.f == -1 else m([x if j == -1 else y[j] for j in m.f])
        y.append(x if m.i in net.save else None)
        if m.i in FEAT_IDX:
            feats[m.i] = x
    return feats

def xray(img_path, bbox_abs, cls_id, neigh=2):
    img = cv2.imread(img_path)
    padded, scale, dw, dh = letterbox(img)
    with torch.no_grad():
        feats = get_feats(to_tensor(padded))
    x1, y1, x2, y2 = bbox_abs
    cx = ((x1 + x2) / 2) * scale + dw
    cy = ((y1 + y2) / 2) * scale + dh
    rows = []
    for k, fi in enumerate(FEAT_IDX):
        pmap = torch.sigmoid(head.one2one_cv3[k](feats[fi]))[0]  # [7, H, W]
        C, H, W = pmap.shape
        stride = int(640 / H)
        gx = min(max(int(cx / stride), 0), W - 1)
        gy = min(max(int(cy / stride), 0), H - 1)
        region = pmap[:, max(0, gy - neigh):gy + neigh + 1, max(0, gx - neig
        per_cls_max = region.flatten(1).max(dim=1).values
        rows.append({
            'k': k, 'layer': fi, 'stride': stride, 'grid': (gx, gy), 'HW': (
            'target_prob': per_cls_max[cls_id].item(),
            'argmax_cls': per_cls_max.argmax().item(),
            'argmax_prob': per_cls_max.max().item(),
        })
    return rows

# ---------- CONTROL (Commandment #4): first glass_shatter (class 3) instanc
control = None
for label_file in sorted(glob.glob(os.path.join(VAL_LABELS_DIR, "*.txt"))):
    with open(label_file) as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) >= 5 and int(parts[0]) == 3:
                coords = list(map(float, parts[1:]))
```

```
                xs, ys = coords[0::2], coords[1::2]
                base = os.path.basename(label_file).replace(".txt", "")
                for ext in [".jpg", ".jpeg", ".png"]:
                    p = os.path.join(VAL_IMAGES_DIR, base + ext)
                    if os.path.exists(p):
                        img0 = cv2.imread(p)
                        H0, W0 = img0.shape[:2]
                        control = {'image': base, 'path': p, 'cls': 3,
                                   'bbox': [int(min(xs) * W0), int(min(ys) *
                                            int(max(xs) * W0), int(max(ys) *
                        break
            if control:
                break
    if control:
        break

print("CONTROL:", control['image'], "| bbox:", control['bbox'])
for r in xray(control['path'], control['bbox'], 3):
    print(f"  scale {r['k']} (layer {r['layer']}, stride {r['stride']}, gric
          f"target P={r['target_prob']:.4f} | argmax cls={r['argmax_cls']} (
```

X-Ray the 7 Missed Cases

In [ ]:
```
print("="*60)
print("X-RAYING THE 7 MISSED CASES")
print("="*60)

for i, case in enumerate(missed_cases):
    cls_id = 5 if case['class_name'] == 'corrosion' else 6
    print(f"\n--- Case {i+1}: {case['image']} | {case['class_name']} (cls {c
    print(f"Original BBox (abs pixels): {case['gt_bbox']}")

    rows = xray(case['path'], case['gt_bbox'], cls_id)
    for r in rows:
        print(f"  scale {r['k']} (stride {r['stride']}, grid {r['grid']}): "
              f"target P={r['target_prob']:.4f} | argmax cls={r['argmax_cls'
```

The SAHI Patch Simulation

In [ ]:
```
import numpy as np

def xray_img(img, bbox_abs, cls_id, neigh=2):
    padded, scale, dw, dh = letterbox(img)
    with torch.no_grad():
        feats = get_feats(to_tensor(padded))
    x1, y1, x2, y2 = bbox_abs
    cx = ((x1 + x2) / 2) * scale + dw
    cy = ((y1 + y2) / 2) * scale + dh
    rows = []
    for k, fi in enumerate(FEAT_IDX):
        pmap = torch.sigmoid(head.one2one_cv3[k](feats[fi]))[0]
```

```
        C, H, W = pmap.shape
        stride = int(640 / H)
        gx = min(max(int(cx / stride), 0), W - 1)
        gy = min(max(int(cy / stride), 0), H - 1)
        region = pmap[:, max(0, gy - neigh):gy + neigh + 1, max(0, gx - neig
        per_cls_max = region.flatten(1).max(dim=1).values
        rows.append((k, stride, per_cls_max[cls_id].item(),
                     per_cls_max.argmax().item(), per_cls_max.max().item()))
    return rows

def extract_patch(img, cx, cy, size=640):
    H0, W0 = img.shape[:2]
    if H0 < size or W0 < size:
        img = cv2.copyMakeBorder(img, 0, max(0, size - H0), 0, max(0, size -
                                 cv2.BORDER_CONSTANT, value=(114, 114, 114))
        H0, W0 = img.shape[:2]
    x1 = min(max(int(round(cx - size / 2)), 0), W0 - size)
    y1 = min(max(int(round(cy - size / 2)), 0), H0 - size)
    return img[y1:y1 + size, x1:x1 + size], x1, y1

print("SAHI PATCH SIMULATION (native-res 640x640 crop)")
for i, case in enumerate(missed_cases):
    cls_id = 5 if case['class_name'] == 'corrosion' else 6
    img = cv2.imread(case['path'])
    x1, y1, x2, y2 = case['gt_bbox']
    patch, px, py = extract_patch(img, (x1 + x2) / 2, (y1 + y2) / 2)
    local_bbox = [x1 - px, y1 - py, x2 - px, y2 - py]
    best = max(xray_img(patch, local_bbox, cls_id), key=lambda r: r[2])
    print(f"Case {i+1} {case['class_name']:>13} {x2-x1:>3}x{y2-y1:<3}px | "
          f"native-patch P={best[2]:.4f} (scale {best[0]}, argmax cls {best[
```

SAHI Verification on Missed Cases

In [ ]:

```
from sahi.predict import get_sliced_prediction
from sahi import AutoDetectionModel
import os

os.makedirs("sahi_preds", exist_ok=True)

id_to_name = {
    0: "dent", 1: "scratch", 2: "crack", 3: "glass_shatter",
    4: "broken_lamp", 5: "corrosion", 6: "disjoint_part"
}

print("Loading SAHI model (this takes a moment)...")
try:
    sahi_model = AutoDetectionModel.from_pretrained(
        model_path=MODEL_PATH,
        model_type="yolov8",  # SAHI uses 'yolov8' for recent ultralytics mo
        device="cuda:0",
        confidence_threshold=0.10,  # Lowered to catch the 0.11 - 0.27 signa
    )
except Exception as e:
    print(f"Failed with 'yolov8', trying 'ultralytics'... Error was: {e}")
```

```
    sahi_model = AutoDetectionModel.from_pretrained(
        model_path=MODEL_PATH,
        model_type="ultralytics",
        device="cuda:0",
        confidence_threshold=0.10,
    )

target_images = [case['path'] for case in missed_cases]

print(f"\nRunning SAHI on {len(target_images)} missed cases (this may take 1
for i, img_path in enumerate(target_images):
    result = get_sliced_prediction(
        img_path,
        sahi_model,
        slice_height=640,
        slice_width=640,
        overlap_height_ratio=0.15,      # 15% overlap (Commandment #6)
        overlap_width_ratio=0.15,
        postprocess_match_metric="IOS", # Intersection over Smaller area (fo
        postprocess_match_threshold=0.50,
    )

    preds = result.object_prediction_list
    print(f"\n--- Case {i+1}: {os.path.basename(img_path)} ---")
    print(f"Total objects found: {len(preds)}")

    for p in preds:
        # Robustly get class ID and name
        try:
            cls_id = int(p.category.id)
            cls_name = id_to_name.get(cls_id, p.category.name)
        except:
            cls_name = p.category.name
            cls_id = -1

        score = p.score.value
        bbox = p.bbox.to_coco_bbox() # [x, y, w, h]

        if cls_name in ["corrosion", "disjoint_part"]:
            print(f"  -> RARE CLASS FOUND: {cls_name} | Score: {score:.3f} |
        elif score > 0.3:
            print(f"     Other high-conf: {cls_name} | Score: {score:.3f}")
```

GT-vs-SAHI Overlay (Visual Verification)

In [ ]:

```
import cv2
import numpy as np

unique_paths = []
for case in missed_cases:
    if case['path'] not in unique_paths:
        unique_paths.append(case['path'])

for img_path in unique_paths:
```

```
    img = cv2.imread(img_path)
    vis = img.copy()

    # --- GT rare-class polygons (GREEN) ---
    label_path = img_path.replace("/images/", "/labels/").rsplit(".", 1)[0]
    if os.path.exists(label_path):
        H0, W0 = img.shape[:2]
        with open(label_path) as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) >= 7 and int(parts[0]) in (5, 6):
                    coords = list(map(float, parts[1:]))
                    pts = np.array([[int(coords[i] * W0), int(coords[i + 1]
                                    for i in range(0, len(coords) - 1, 2)],
                    cv2.polylines(vis, [pts], True, (0, 255, 0), 2)

    # --- SAHI predictions ---
    result = get_sliced_prediction(
        img_path, sahi_model,
        slice_height=640, slice_width=640,
        overlap_height_ratio=0.15, overlap_width_ratio=0.15,
        postprocess_match_metric="IOS", postprocess_match_threshold=0.50,
    )
    for p in result.object_prediction_list:
        score = p.score.value
        try:
            cls_id = int(p.category.id)
        except Exception:
            cls_id = -1
        x, y, w, h = [int(v) for v in p.bbox.to_coco_bbox()]
        if cls_id in (5, 6) and score >= 0.10:
            cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 0, 255), 2)
            cv2.putText(vis, f"{p.category.name} {score:.2f}", (x, max(12, y
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
        elif score >= 0.30:
            cv2.rectangle(vis, (x, y), (x + w, y + h), (255, 0, 0), 1)

    out = os.path.join("sahi_preds", os.path.basename(img_path).rsplit(".",
    cv2.imwrite(out, vis)
    print("Saved", out)
```

Our Own Post-Processing + Fixed Overlays

In [ ]:

```
def ios(a, b):
    ax1, ay1, ax2, ay2 = a
    bx1, by1, bx2, by2 = b
    iw = min(ax2, bx2) - max(ax1, bx1)
    ih = min(ay2, by2) - max(ay1, by1)
    if iw <= 0 or ih <= 0:
        return 0.0
    return (iw * ih) / min((ax2 - ax1) * (ay2 - ay1), (bx2 - bx1) * (by2 - b

def our_postprocess(dets, ios_thresh=0.5, min_area=30):
    dets = [d for d in dets if (d['bbox'][2] - d['bbox'][0]) * (d['bbox'][3]
```

```
    dets.sort(key=lambda d: -d['score'])
    kept = []
    for d in dets:
        if not any(d['cls'] == k['cls'] and ios(d['bbox'], k['bbox']) >= ios
            kept.append(d)
    return kept

for img_path in unique_paths:
    img = cv2.imread(img_path)
    vis = img.copy()

    # GT rare polygons (GREEN)
    label_path = img_path.replace("/images/", "/labels/").rsplit(".", 1)[0]
    if os.path.exists(label_path):
        H0, W0 = img.shape[:2]
        with open(label_path) as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) >= 7 and int(parts[0]) in (5, 6):
                    coords = list(map(float, parts[1:]))
                    pts = np.array([[int(coords[i] * W0), int(coords[i + 1]
                                    for i in range(0, len(coords) - 1, 2)],
                    cv2.polylines(vis, [pts], True, (0, 255, 0), 2)

    # SAHI raw detections
    result = get_sliced_prediction(
        img_path, sahi_model,
        slice_height=640, slice_width=640,
        overlap_height_ratio=0.15, overlap_width_ratio=0.15,
    )
    dets = []
    for p in result.object_prediction_list:
        try:
            cls_id = int(p.category.id)
        except Exception:
            continue
        x, y, w, h = [int(v) for v in p.bbox.to_coco_bbox()]
        dets.append({'cls': cls_id, 'score': p.score.value,
                     'name': p.category.name, 'bbox': [x, y, x + w, y + h]})

    kept = our_postprocess(dets)
    print(f"{os.path.basename(img_path)}: {len(dets)} raw -> {len(kept)} aft

    for d in kept:
        x1, y1, x2, y2 = d['bbox']
        if d['cls'] in (5, 6) and d['score'] >= 0.10:
            cv2.rectangle(vis, (x1, y1), (x2, y2), (0, 0, 255), 2)
            cv2.putText(vis, f"{d['name']} {d['score']:.2f}", (x1, max(12, y
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
        elif d['score'] >= 0.30:
            cv2.rectangle(vis, (x1, y1), (x2, y2), (255, 0, 0), 1)

    out = os.path.join("sahi_preds", os.path.basename(img_path).rsplit(".",
    cv2.imwrite(out, vis)
    print("Saved", out)
```

Mask Introspection

In [ ]:
```
result = get_sliced_prediction(
    unique_paths[0], sahi_model,
    slice_height=640, slice_width=640,
    overlap_height_ratio=0.15, overlap_width_ratio=0.15,
)
p = result.object_prediction_list[0]
print("ObjectPrediction attrs:", [a for a in dir(p) if not a.startswith('_')
print("mask type:", type(p.mask))
if p.mask is not None:
    print("mask attrs:", [a for a in dir(p.mask) if not a.startswith('_')])
    try:
        bm = p.mask.bool_mask
        print("bool_mask shape:", bm.shape, "| dtype:", bm.dtype, "| pixels:
    except Exception as e:
        print("mask access issue:", e)
```

Mask-IOS NMS + Per-Class Rules + Polygon Overlays

In [ ]:
```
def mask_ios(a_mask, a_area, b_mask, b_area):
    inter = int(np.logical_and(a_mask, b_mask).sum())
    smaller = min(a_area, b_area)
    return inter / smaller if smaller > 0 else 0.0

def accepted(cls_id, score, area):
    if cls_id == 5:    # corrosion: recall priority
        return score >= 0.10 and area >= 30
    if cls_id == 6:    # disjoint_part: precision priority
        return score >= 0.25 and area >= 200
    return score >= 0.30

for img_path in unique_paths:
    img = cv2.imread(img_path)
    vis = img.copy()

    # GT rare polygons (GREEN)
    label_path = img_path.replace("/images/", "/labels/").rsplit(".", 1)[0]
    if os.path.exists(label_path):
        H0, W0 = img.shape[:2]
        with open(label_path) as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) >= 7 and int(parts[0]) in (5, 6):
                    coords = list(map(float, parts[1:]))
                    pts = np.array([[int(coords[i] * W0), int(coords[i + 1]
                                    for i in range(0, len(coords) - 1, 2)],
                    cv2.polylines(vis, [pts], True, (0, 255, 0), 2)
```

```
    result = get_sliced_prediction(
        img_path, sahi_model,
        slice_height=640, slice_width=640,
        overlap_height_ratio=0.15, overlap_width_ratio=0.15,
    )

    dets = []
    for p in result.object_prediction_list:
        try:
            cls_id = int(p.category.id)
        except Exception:
            continue
        m = (np.asarray(p.mask.bool_mask) > 0.5)
        area = int(m.sum())
        if not accepted(cls_id, p.score.value, area):
            continue
        ys, xs = np.nonzero(m)
        dets.append({'cls': cls_id, 'score': p.score.value, 'name': p.categ
                     'mask': m, 'area': area,
                     'bbox': [int(xs.min()), int(ys.min()), int(xs.max()), i

    # Greedy per-class mask-IOS NMS
    dets.sort(key=lambda d: -d['score'])
    kept = []
    for d in dets:
        dup = False
        for k in kept:
            if d['cls'] != k['cls'] or ios(d['bbox'], k['bbox']) <= 0:
                continue
            if mask_ios(d['mask'], d['area'], k['mask'], k['area']) >= 0.5:
                dup = True
                break
        if not dup:
            kept.append(d)

    print(f"{os.path.basename(img_path)}: {len(result.object_prediction_list
          f"{len(dets)} after per-class rules -> {len(kept)} after mask-IOS

    for d in kept:
        mu8 = d['mask'].astype(np.uint8) * 255
        cnts, _ = cv2.findContours(mu8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_
        if d['cls'] in (5, 6):
            cv2.drawContours(vis, cnts, -1, (0, 0, 255), 2)
            cv2.putText(vis, f"{d['name']} {d['score']:.2f}", (d['bbox'][0],
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
        else:
            cv2.drawContours(vis, cnts, -1, (255, 0, 0), 1)

    out = os.path.join("sahi_preds", os.path.basename(img_path).rsplit(".",
    cv2.imwrite(out, vis)
    print("Saved", out)
```

Equivalence Test

In [ ]:
```
import importlib.util

spec = importlib.util.spec_from_file_location("sahi_inference", "src/stage2/
si = importlib.util.module_from_spec(spec)
spec.loader.exec_module(si)

legacy_cfg = {
    "preset": "legacy",
    "model_type": "yolov8",
    "sahi": {"slice_size": 640, "overlap_ratio": 0.15},
    "nms": {"ios_threshold": 0.5},
    "presets": {"legacy": {"class_rules": {
        "0": {"conf": 0.30, "min_area": 0}, "1": {"conf": 0.30, "min_area":
        "2": {"conf": 0.30, "min_area": 0}, "3": {"conf": 0.30, "min_area":
        "4": {"conf": 0.30, "min_area": 0}, "5": {"conf": 0.10, "min_area":
        "6": {"conf": 0.25, "min_area": 200},
    }}},
}

engine = si.SahiInference(MODEL_PATH, legacy_cfg)

os.makedirs("sahi_preds/module_check", exist_ok=True)
for img_path in unique_paths:
    preds = engine.predict(img_path)
    n_rare = len([p for p in preds if p["cls"] in (5, 6)])
    print(f"{os.path.basename(img_path)}: {len(preds)} kept ({n_rare} rare)"
    vis = engine.visualize(img_path, preds,
                           gt_label_path=img_path.replace("/images/", "/labe
    cv2.imwrite(os.path.join("sahi_preds/module_check", os.path.basename(img
```

Collect Low-Conf Predictions + GT Masks

In [ ]:
```
import importlib.util

REPO_ROOT = "/home/lamacpp/Documents/car_defect_detection"
MOD_PATH = os.path.join(REPO_ROOT, "src/stage2/inference/sahi_inference.py")
print("module exists:", os.path.exists(MOD_PATH))
spec = importlib.util.spec_from_file_location("sahi_inference", MOD_PATH)
si = importlib.util.module_from_spec(spec)
spec.loader.exec_module(si)

cfg = si.load_config(os.path.join(REPO_ROOT, "configs/inference/sahi_product

cfg["preset"] = "calib"
engine_cal = si.SahiInference(MODEL_PATH, cfg)

calib = []
for idx, (img_name, gt_data) in enumerate(gt_targets.items()):
    img = cv2.imread(gt_data['path'])
    H0, W0 = img.shape[:2]
    gt_masks = []
    label_path = gt_data['path'].replace("/images/", "/labels/").rsplit(".",
    with open(label_path) as f:
```

```
        for line in f:
            parts = line.strip().split()
            if len(parts) >= 7 and int(parts[0]) in (5, 6):
                coords = list(map(float, parts[1:]))
                pts = np.array([[int(coords[i] * W0), int(coords[i + 1] * H0
                                for i in range(0, len(coords) - 1, 2)], np.i
                m = np.zeros((H0, W0), np.uint8)
                cv2.fillPoly(m, [pts], 1)
                gt_masks.append({'cls': int(parts[0]), 'mask': m.astype(bool

    preds = [p for p in engine_cal.predict(gt_data['path']) if p['cls'] in (
    calib.append({'img': img_name, 'gt': gt_masks, 'pred': preds})
    if (idx + 1) % 10 == 0:
        print(f"  collected {idx + 1}/{len(gt_targets)} ...")

n_p5 = sum(len([p for p in c['pred'] if p['cls'] == 5]) for c in calib)
n_p6 = sum(len([p for p in c['pred'] if p['cls'] == 6]) for c in calib)
print(f"kept preds @1024: corrosion={n_p5}, disjoint={n_p6}")
```

Threshold Sweep

In [ ]:
```
def eval_thresh(cls_id, thresh):
    tp = fp = fn = 0
    for c in calib:
        gts = [g for g in c['gt'] if g['cls'] == cls_id]
        used = [False] * len(gts)
        for p in [p for p in c['pred'] if p['cls'] == cls_id and p['score']
            best_i, best_v = -1, 0.0
            for j, g in enumerate(gts):
                if used[j]:
                    continue
                v = mask_ios(p['mask'], p['area'], g['mask'], g['area'])
                if v > best_v:
                    best_i, best_v = j, v
            if best_i >= 0 and best_v >= 0.5:
                used[best_i] = True
                tp += 1
            else:
                fp += 1
        fn += sum(1 for u in used if not u)
    prec = tp / (tp + fp) if (tp + fp) else 0.0
    rec = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2 * prec * rec / (prec + rec) if (prec + rec) else 0.0
    return prec, rec, f1, tp, fp, fn

for cls_id, name in [(5, 'corrosion'), (6, 'disjoint_part')]:
    print(f"\n=== {name} ===")
    print(" thresh | precision | recall |  F1  | TP FP FN")
    for t in np.arange(0.05, 0.501, 0.05):
        prec, rec, f1, tp, fp, fn = eval_thresh(cls_id, round(float(t), 2))
        print(f"  {t:.2f}  |   {prec:.3f}   | {rec:.3f} | {f1:.3f} | {tp:>2}
```

In [ ]:
```
!jupyter nbconvert --to webpdf --allow-chromium-download "phase3_diagnostics
```

In [ ]: